wojteklu / clean_code.md
Last active yesterday • Report abuse

Code Revisions 2 Stars 5,000+ Forks 1,433 Embed Download ZIP

Summary of 'Clean code' by Robert C. Martin

clean_code.md

Code is clean if it can be understood easily – by everyone on the team. Clean code can be read and enhanced by a developer other than its original author. With understandability comes readability, changeability, extensibility and maintainability.

General rules

1. Follow standard conventions.
2. Keep it simple stupid. Simpler is always better. Reduce complexity as much as possible.
3. Boy scout rule. Leave the campground cleaner than you found it.
4. Always find root cause. Always look for the root cause of a problem.

Design rules

1. Keep configurable data at high levels.
2. Prefer polymorphism to if/else or switch/case.
3. Separate multi-threading code.
4. Prevent over-configurability.
5. Use dependency injection.
6. Follow Law of Demeter. A class should know only its direct dependencies.

Understandability tips

1. Be consistent. If you do something a certain way, do all similar things in the same way.
2. Use explanatory variables.
3. Encapsulate boundary conditions. Boundary conditions are hard to keep track of. Put the processing for them in one place.
4. Prefer dedicated value objects to primitive type.
5. Avoid logical dependency. Don't write methods which works correctly depending on something else in the same class.
6. Avoid negative conditionals.

Names rules

1. Choose descriptive and unambiguous names.
2. Make meaningful distinction.
3. Use pronounceable names.
4. Use searchable names.
5. Replace magic numbers with named constants.
6. Avoid encodings. Don't append prefixes or type information.

Functions rules

1. Small.
2. Do one thing.
3. Use descriptive names.
4. Prefer fewer arguments.
5. Have no side effects.
6. Don't use flag arguments. Split method into several independent methods that can be called from the client without the flag.

Comments rules

1. Always try to explain yourself in code.
2. Don't be redundant.
3. Don't add obvious noise.
4. Don't use closing brace comments.
5. Don't comment out code. Just remove.
6. Use as explanation of intent.
7. Use as clarification of code.
8. Use as warning of consequences.

Source code structure

1. Separate concepts vertically.
2. Related code should appear vertically dense.
3. Declare variables close to their usage.
4. Dependent functions should be close.
5. Similar functions should be close.
6. Place functions in the downward direction.
7. Keep lines short.
8. Don't use horizontal alignment.
9. Use white space to associate related things and disassociate weakly related.

10. Don't break indentation.

Objects and data structures

1. Hide internal structure.
2. Prefer data structures.
3. Avoid hybrids structures (half object and half data).
4. Should be small.
5. Do one thing.
6. Small number of instance variables.
7. Base class should know nothing about their derivatives.
8. Better to have many functions than to pass some code into a function to select a behavior.
9. Prefer non-static methods to static methods.

Tests

1. One assert per test.
2. Readable.
3. Fast.
4. Independent.
5. Repeatable.

Code smells

1. Rigidity. The software is difficult to change. A small change causes a cascade of subsequent changes.
2. Fragility. The software breaks in many places due to a single change.
3. Immobility. You cannot reuse parts of the code in other projects because of involved risks and high effort.
4. Needless Complexity.
5. Needless Repetition.
6. Opacity. The code is hard to understand.

Load earlier comments...

JohannesJacop commented on Aug 29, 2023

I am not a programmer myself. So, I haven't read the book myself (although I knew about it for sure 😉) and I read the summary here now for the first time—and I just realized, the recommendations actually apply to quite a lot of other things as well!

So, for all non-engineers, it's worth the read! 💡 Thanks for sharing. 👍

pedroluiznogueira commented on Sep 2, 2023

> > > > I would like to contribute to this amazing Clean Code hints by one that I noticed it's possible to be done and nobody talks about...
> > > >
> > > > ## Conditional Simplification
> > > >
> > > > You can always turn a complex conditional into a simpler one through the Propositional Logic rules
> > > >
> > > > app/javascript/packs/ui/core/grid/editable_grid.jsx
> > > >
> > > > ```
> > > > 214 -  const setting = fieldSettings[column.foreignAttribute] || fieldSettings[column.descrip
> > > > 215 -  return !(column.hideOnGrid || (setting && setting.hide_on_grid))
> > > > 214 +  const setting = fieldSettings[column.foreignAttribute] || fieldSettings[column.descrip
> > > > 215 +  return !column.hideOnGrid && !setting.hide_on_grid
> > > > ```
> > > >
> > > > **Victorcorcos** 1 hour ago Author
> > > >
> > > > `!(p || (q && r))` 👉 `!p && !r`
> > > >
> > > > **Victorcorcos** 27 minutes ago • edited Author
> > > >
> > > > **Before**
> > > >
> > > > ```
> > > > !(p || (q && r))
> > > > # 💬 Show the column if NOT (hide on grid on column OR (setting is available AND NOT hide on grid on
> > > > ```
> > > >
> > > > **Simplification**
> > > >
> > > > 1. Avoid NPE on `q` by using `|| {}`
> > > > 2. `!(p || r)` (removes `q`, because Tautology)
> > > > 3. `!p && !r` (De Morgan)
> > > >
> > > > **Result**
> > > >
> > > > ```
> > > > !p && !r
> > > > # 💬 Show the column if NOT hide on grid on column AND NOT hide on grid on setting
> > > > ```
> > > >
> > > > - References
> > > >
> > > > 1. De Morgan
> > > > 2. Tautology
> > > > 3. Propositional Logic
> > > >
> > > > Inspired by Robert C Marin's book: Clean Code
> >
> > Or you could just paste an "obfuscated" ((a && b) || (b &&c) || (!a && !c)) version of your condition into https://www.wolframalpha.com/ and let it do the work. (Minimal forms)
>
> That's work. But if "setting" is null when hideOnGrid is false, it's generate exception.

You could define additional vars to add semantic. Ex.

```
const mustHideOnGridByColumn = column.hideOnGrid;
const mustHideOnGridBySetting = setting && setting.hide_on_grid;

return !(mustHideOnGridByColumn || mustHideOnGridBySetting);
 // or: return !mustHideOnGridByColumn && !mustHideOnGridBySetting
```

This way is very clean and improves readability

This is by far one of the best things I've started doing for a while.

Usually boolean conditions are a representation of a behavior, meaningful names will always help your code read like a story.

That way just by reading the variable name you get the problem statement.

For example:

if (isReadyToReceiveMessage) {...}

Of course one should be careful to not create misleading variables that don't mean what the condition represent. (tests help with that semantics).

Victorcorcos commented on Sep 12, 2023 • edited

> @Victorcorcos if `setting` is `null` or `undefined` the old code is working but not the new code, or am I missing something ?

The new code will work, because this is applied:

1. Avoid NPE on `q` by using `|| {}`

- Before

```
setting = fieldSettings[column.foreignAttribute] || fieldSettings[column.description]
```

- After

```
setting = fieldSettings[column.foreignAttribute] || fieldSettings[column.description] || {}
```

So, in case `setting` is `null` / `undefined`, it will be `{}`, which won't cause NPE.

→ We can't see that on the picture, but that `|| {}` was the change on that specific line

SpringfieldC commented on Oct 22, 2023

Interesting

M1SHANIA commented on Dec 14, 2023

Can I download this book somewhere for free? TY<3

philipvella commented on Dec 14, 2023

thanks for this @wojteklu 👍

Dariushdragon commented on Jan 14, 2024

it helps a lot for me thank you.

hassan-mohagheghian commented on Jan 26, 2024

sounds great, thank you!

arkadiusjonczek commented on Jan 28, 2024

❤️

jjsutil commented on May 4, 2024

Thanks for this notes!

fosterushka commented on Jun 14, 2024

Would be helpful if someone will add code example refactored as above one.

realgio95 commented on Jul 18, 2024

Adding Some Code examples in C# with ChatGPT

Code Examples with GPT4o

Certainly! Here are C# code examples illustrating the principles and rules described:

General rules

1. **Follow standard conventions**

```
// Correct
public class OrderService
{
    private readonly IOrderRepository _orderRepository;

    public OrderService(IOrderRepository orderRepository)
    {
        _orderRepository = orderRepository;
```

```
    }

    public void PlaceOrder(Order order)
    {
        _orderRepository.Add(order);
    }
}

// Incorrect
public class orderservice
{
    private IOrderRepository orderrepository;

    public orderservice(IOrderRepository orderrepository)
    {
        this.orderrepository = orderrepository;
    }

    public void placeorder(Order order)
    {
        orderrepository.add(order);
    }
}
```

2. **Keep it simple stupid. Simpler is always better. Reduce complexity as much as possible**

```
// Simple and clear
public double CalculateArea(double radius)
{
    return Math.PI * radius * radius;
}

// Overly complex
public double CalculateArea(double radius)
{
    double area = 0;
    for (int i = 0; i < 360; i++)
    {
        area += (Math.PI / 180) * radius * radius;
    }
    return area;
}
```

3. **Boy scout rule. Leave the campground cleaner than you found it**

```
// Clean up resources
public void ProcessData()
{
    using (var connection = new SqlConnection(connectionString))
    {
        connection.Open();
        // Perform data operations
    } // Connection is automatically closed here
}
```

4. **Always find root cause. Always look for the root cause of a problem**

```
// Logging the exception and identifying the root cause
public void ProcessOrder(Order order)
```

```
{
    try
    {
        // Process the order
    }
    catch (Exception ex)
    {
        Log.Error("Error processing order: " + ex.Message);
        throw;
    }
}
```

Design rules

1. **Keep configurable data at high levels**

```
// Configurable data at high level
public class Config
{
    public string DatabaseConnectionString { get; set; }
}

public class DataService
{
    private readonly string _connectionString;

    public DataService(Config config)
    {
        _connectionString = config.DatabaseConnectionString;
    }

    // ...
}
```

2. **Prefer polymorphism to if/else or switch/case**

```
// Using polymorphism
public abstract class Notification
{
    public abstract void Send();
}

public class EmailNotification : Notification
{
    public override void Send()
    {
        // Send email
    }
}

public class SmsNotification : Notification
{
    public override void Send()
    {
        // Send SMS
    }
}

// Instead of
```

```
public void SendNotification(string type)
{
    if (type == "Email")
    {
        // Send email
    }
    else if (type == "SMS")
    {
        // Send SMS
    }
}
```

3. **Separate multi-threading code**

```
// Using Task for multi-threading
public async Task ProcessDataAsync()
{
    await Task.Run(() =>
    {
        // Perform CPU-bound operations
    });
}
```

4. **Prevent over-configurability**

```
// Simple configuration
public class AppSettings
{
    public string DatabaseConnectionString { get; set; }
}
```

5. **Use dependency injection**

```
// Using dependency injection
public class OrderService
{
    private readonly IOrderRepository _orderRepository;

    public OrderService(IOrderRepository orderRepository)
    {
        _orderRepository = orderRepository;
    }

    // ...
}
```

6. **Follow Law of Demeter. A class should know only its direct dependencies**

```
// Following Law of Demeter
public class OrderProcessor
{
    private readonly OrderService _orderService;

    public OrderProcessor(OrderService orderService)
    {
        _orderService = orderService;
    }
```

```
    public void Process(Order order)
    {
        _orderService.PlaceOrder(order);
    }
}
```

Understandability tips

1. **Be consistent**

```
// Consistent naming
public class UserService
{
    public User GetUserById(int id)
    {
        // ...
    }

    public User GetUserByEmail(string email)
    {
        // ...
    }
}
```

2. **Use explanatory variables**

```
// Using explanatory variables
var fileName = "data.txt";
var fileContent = File.ReadAllText(fileName);
```

3. **Encapsulate boundary conditions**

```
// Encapsulating boundary conditions
public class DateRange
{
    public DateTime Start { get; }
    public DateTime End { get; }

    public DateRange(DateTime start, DateTime end)
    {
        if (start > end)
            throw new ArgumentException("Start date must be earlier than end date");

        Start = start;
        End = end;
    }
}
```

4. **Prefer dedicated value objects to primitive type**

```
// Using value objects
public class Email
{
    public string Address { get; }
```

```
    public Email(string address)
    {
        if (string.IsNullOrEmpty(address))
            throw new ArgumentException("Email address cannot be empty");

        Address = address;
    }
}
```

5. **Avoid logical dependency**

```
// Avoiding logical dependency
public class User
{
    public string Name { get; set; }
    public bool IsActive { get; set; }

    public void Activate()
    {
        IsActive = true;
    }
}
```

6. **Avoid negative conditionals**

```
// Avoiding negative conditionals
if (IsUserActive)
{
    // Do something
}

// Instead of
if (!IsUserInactive)
{
    // Do something
}
```

Names rules

1. **Choose descriptive and unambiguous names**

```
// Descriptive names
public class InvoiceService
{
    public void CreateInvoice(Order order)
    {
        // ...
    }
}
```

2. **Make meaningful distinction**

```
// Meaningful distinction
public class Order
{
    public DateTime OrderDate { get; set; }
```

```
}

public class Shipment
{
    public DateTime ShipmentDate { get; set; }
}
```

3. **Use pronounceable names**

```
// Pronounceable names
public class CustomerService
{
    public void AddCustomer(Customer customer)
    {
        // ...
    }
}
```

4. **Use searchable names**

```
// Searchable names
public const int MaxUsers = 100;
```

5. **Replace magic numbers with named constants**

```
// Using named constants
public const int DefaultTimeout = 30;

public void Connect()
{
    // Using named constant
    var timeout = DefaultTimeout;
}
```

6. **Avoid encodings**

```
// Avoiding encodings
public class UserService
{
    public void CreateUser(string name)
    {
        // ...
    }
}
```

Functions rules

1. **Small**

```
// Small function
public void SaveUser(User user)
{
    ValidateUser(user);
```

```
    _userRepository.Save(user);
}
```

2. **Do one thing**

```
// Single responsibility
public void ValidateUser(User user)
{
    if (string.IsNullOrEmpty(user.Name))
        throw new ArgumentException("User name cannot be empty");
}
```

3. **Use descriptive names**

```
// Descriptive names
public void SendEmailConfirmation(User user)
{
    // ...
}
```

4. **Prefer fewer arguments**

```
// Fewer arguments
public void CreateOrder(int userId, int productId)
{
    // ...
}
```

5. **Have no side effects**

```
// No side effects
public int CalculateSum(int a, int b)
{
    return a + b;
}
```

6. **Don't use flag arguments**

```
// Separate methods
public void EnableFeature()
{
    // ...
}

public void DisableFeature()
{
    // ...
}
```

Comments rules

1. **Always try to explain yourself in code**

```
// Self-explanatory code
public void CalculateTotalPrice()
{
    var totalPrice = items.Sum(item => item.Price * item.Quantity);
}
```

2. **Don't be redundant**

```
// Redundant comment
// Adds two numbers
public int Add(int a, int b)
{
    return a + b;
}
```

3. **Don't add obvious noise**

```
// Obvious noise
public int Multiply(int a, int b)
{
    return a * b; // Multiply a and b
}
```

4. **Don't use closing brace comments**

```
// No closing brace comments
public void ProcessOrder()
{
    // ...
}
```

5. **Don't comment out code. Just remove**

```
// Remove commented code
// public void OldMethod()
// {
//     // ...
// }
```

6. **Use as explanation of intent**

```
// Explanation of intent
public void SaveData()
{
    // Save data to ensure it is not lost
    _dataRepository.Save();
}
```

7. **Use as clarification of code**

```
// Clarification of code
public void CalculateDiscount()
```

```
{
    // Apply 10% discount to all items
    foreach (var item in items)
    {
        item.Price *= 0.9;
    }
}
```

8. **Use as warning of consequences**

```csharp
// Warning of consequences
public void DeleteUser(int userId)
{
    // Warning: This action is irreversible
    _userRepository.Delete(userId);
}
```

Source code structure

1. **Separate concepts vertically**

```
// Vertical separation
public class UserService
{
    // Constructor
    public UserService(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    // Public methods
    public void CreateUser(User user)
    {
        // ...
    }

    // Private methods
    private void ValidateUser(User user)
    {
        // ...
    }
}
```

2. **Related code should appear vertically dense**

```
// Related code close together
public class OrderService
{
    public void CreateOrder(Order order)
    {
        ValidateOrder(order);
        SaveOrder(order);
    }

    private void ValidateOrder(Order order)
    {
```

```
        // ...
    }

    private void SaveOrder(Order order)
    {
        // ...
    }
}
```

3. **Declare variables close to their usage**

```
public void ProcessOrder(Order order)
{
    var orderTotal = CalculateTotal(order); // Variable declared close to usage
    // ...
}
```

4. **Dependent functions should be close**

```
public class OrderService
{
    public void CreateOrder(Order order)
    {
        var total = CalculateTotal(order);
        SaveOrder(order, total);
    }

    private decimal CalculateTotal(Order order)
    {
        // ...
    }

    private void SaveOrder(Order order, decimal total)
    {
        // ...
    }
}
```

5. **Similar functions should be close**

```
public class UserService
{
    public User GetUserById(int id)
    {
        // ...
    }

    public User GetUserByEmail(string email)
    {
        // ...
    }
}
```

6. **Place functions in the downward direction**

```
public class OrderService
{
```

```
    public void ProcessOrder(Order order)
    {
        ValidateOrder(order);
        SaveOrder(order);
    }

    private void ValidateOrder(Order order)
    {
        // ...
    }

    private void SaveOrder(Order order)
    {
        // ...
    }
}
```

7. **Keep lines short**

```
// Short lines
public void DisplayUserInfo(User user)
{
    Console.WriteLine($"Name: {user.Name}, Email: {user.Email}");
}
```

8. **Don't use horizontal alignment**

```
// No horizontal alignment
public int MaxUsers    { get; set; }
public string Database { get; set; }
```

9. **Use white space to associate related things and disassociate weakly related**

```
// Using white space
public class UserService
{
    public void CreateUser(User user)
    {
        ValidateUser(user);
        _userRepository.Save(user);
    }

    private void ValidateUser(User user)
    {
        if (string.IsNullOrEmpty(user.Name))
            throw new ArgumentException("Name is required");
    }
}
```

10. **Don't break indentation**

```
// Correct indentation
public void ProcessData()
{
    if (data.IsValid)
    {
        SaveData(data);
```

```
    }
}
```

Objects and data structures

1. **Hide internal structure**

```
// Encapsulation
public class Order
{
    private List<OrderItem> _items;

    public void AddItem(OrderItem item)
    {
        _items.Add(item);
    }

    public decimal GetTotal()
    {
        return _items.Sum(i => i.Price * i.Quantity);
    }
}
```

2. **Prefer data structures**

```
// Data structure
public class Point
{
    public int X { get; set; }
    public int Y { get; set; }
}
```

3. **Avoid hybrids structures (half object and half data)**

```
// Avoid hybrids
public class User
{
    private string passwordHash;

    public string Name { get; set; }

    public void SetPassword(string password)
    {
        passwordHash = Hash(password);
    }

    private string Hash(string input)
    {
        // Hashing logic
    }
}
```

4. **Should be small**

```
// Small class
public class Address
```

```
{
    public string Street { get; set; }
    public string City { get; set; }
    public string ZipCode { get; set; }
}
```

5. **Do one thing**

```
// Single responsibility
public class EmailValidator
{
    public bool IsValid(string email)
    {
        // Validation logic
    }
}
```

6. **Small number of instance variables**

```
// Few instance variables
public class Product
{
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

7. **Base class should know nothing about their derivatives**

```
// Proper inheritance
public abstract class Animal
{
    public abstract void MakeSound();
}

public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Bark");
    }
}
```

8. **Better to have many functions than to pass some code into a function to select a behavior**

```
// Many functions
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Subtract(int a, int b)
    {
        return a - b;
```

```
    }
}
```

9. **Prefer non-static methods to static methods**

```
// Non-static method
public class Logger
{
    public void Log(string message)
    {
        // Log message
    }
}
```

Tests

1. **One assert per test**

```
[Test]
public void TestAddition()
{
    var calculator = new Calculator();
    var result = calculator.Add(2, 3);
    Assert.AreEqual(5, result);
}
```

2. **Readable**

```
// Readable test
[Test]
public void ShouldReturnTrueForValidEmail()
{
    var emailValidator = new EmailValidator();
    var isValid = emailValidator.IsValid("test@example.com");
    Assert.IsTrue(isValid);
}
```

3. **Fast**

```
// Fast test
[Test]
public void ShouldProcessOrderQuickly()
{
    var orderProcessor = new OrderProcessor();
    var stopwatch = Stopwatch.StartNew();
    orderProcessor.Process(order);
    stopwatch.Stop();
    Assert.Less(stopwatch.ElapsedMilliseconds, 1000);
}
```

4. **Independent**

```
// Independent test
[Test]
```

```
public void ShouldSaveUser()
{
    var userRepository = new UserRepository();
    var userService = new UserService(userRepository);
    var user = new User { Name = "John Doe" };

    userService.SaveUser(user);

    var savedUser = userRepository.GetUserByName("John Doe");
    Assert.AreEqual(user.Name, savedUser.Name);
}
```

5. **Repeatable**

```
// Repeatable test
[Test]
public void ShouldCalculateTotalPrice()
{
    var cart = new ShoppingCart();
    cart.AddItem(new Product { Price = 10, Quantity = 2 });
    cart.AddItem(new Product { Price = 5, Quantity = 3 });

    var totalPrice = cart.CalculateTotalPrice();

    Assert.AreEqual(35, totalPrice);
}
```

Code smells

1. **Rigidity**

```
// Rigid code example
public class OrderProcessor
{
    public void ProcessOnlineOrder(OnlineOrder order)
    {
        // ...
    }

    public void ProcessInStoreOrder(InStoreOrder order)
    {
        // ...
    }
}
```

2. **Fragility**

```
// Fragile code example
public class User
{
    public string Name { get; set; }
    public string Email { get; set; }
}

public class UserService
{
    private readonly List<User> _users;
```

```
    public void UpdateUserEmail(string name, string newEmail)
    {
        var user = _users.FirstOrDefault(u => u.Name == name);
        if (user != null)
        {
            user.Email = newEmail;
        }
    }
}
```

3. **Immobility**

```
// Immobile code example
public class ReportGenerator
{
    private readonly Database _database;

    public ReportGenerator(Database database)
    {
        _database = database;
    }

    public void GenerateReport()
    {
        var data = _database.GetData();
        // Generate report using data
    }
}
```

4. **Needless Complexity**

```
// Needless complexity
public class ComplexCalculator
{
    public int Calculate(int a, int b, string operation)
    {
        if (operation == "add")
        {
            return a + b;
        }
        else if (operation == "subtract")
        {
            return a - b;
        }
        else if (operation == "multiply")
        {
            return a * b;
        }
        else if (operation == "divide")
        {
            return a / b;
        }
        else
        {
            throw new InvalidOperationException("Unknown operation");
        }
    }
}
```

5. **Needless Repetition**

```
// Needless repetition
public class UserService
{
    private readonly IUserRepository _userRepository;

    public UserService(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public void CreateUser(User user)
    {
        if (string.IsNullOrEmpty(user.Name))
            throw new ArgumentException("Name ```csharp
            throw new ArgumentException("Name is required");

        if (string.IsNullOrEmpty(user.Email))
            throw new ArgumentException("Email is required");

        _userRepository.Save(user);
    }
}
```

6. **Opacity**

```
// Opaque code example
public class InvoiceService
{
    public void ProcessInvoice(Invoice invoice)
    {
        if (invoice != null)
        {
            if (invoice.Amount > 0)
            {
                if (invoice.Status == "Pending")
                {
                    // Process the invoice
                }
            }
        }
    }
}
```

By applying these principles and rules in your C# code, you can enhance readability, maintainability, and overall code quality.

zleypner commented on Aug 24, 2024

Thank you! Very useful.

shubhamprakash911 commented on Aug 31, 2024

thanks for sharing this

Richy55-max commented on Sep 17, 2024

Thank you! Very helpful.

muhammetaydinn commented on Oct 3, 2024

Thanks

Victorcorcos commented on Oct 20, 2024

I created a Github Repository not only with these Clean Code techniques, but also practical didactic code examples explaining how each technique can be applied applied, showing the clear readability difference before and after applying these Clean Code techniques in the code ✨

Here it is!

→ https://github.com/victorcorcos/clean-code

Happy learning! 📚

Let's make a change in programming world by applying Clean Code 🧹

rushout09 commented on Nov 11, 2024

Checkout the top ai code quality tools which can save hours of time:

https://www.codeant.ai/blogs/best-code-quality-tools

Also checkout the code quality metrics to track:

https://www.codeant.ai/blogs/code-quality-metrics-to-track

wojciesh commented on Nov 23, 2024

Hi @wojteklu , I've made some minor language fixes to your gist. You can view my updated version here: https://gist.github.com/wojciesh/39cd8ae881895d8a32c44e1f1bf6d605

boluocat commented on Dec 19, 2024

Thank you very much. It's a good guideline.

Risyandi commented on Mar 14, 2025

Nice, Thank you for the summary.

AtalaySezen commented on May 2, 2025

Özet için teşekkürler.

tolgayilmaz86 commented on May 17, 2025

By experience, no matter how clean your code is it will be eventually outdated or modified. We only try to reduce number of "WTF" moments for our successors. So do not burn out your devs with small details, keep the balance and freedom.

iapostol commented on Jun 14, 2025

Thanks!

mritunjaymusale commented on Jul 11, 2025

I wonder if we feed this gist in to vscode's copilot's chatmode will it rewrite the code in clean code fashion 🤔

legalbirdlitigation commented on Oct 9, 2025

Great content of the book. But I would still like to read it in its entirety.

MoserMichael commented on Jan 1

Great write-up! Did you try to use it as a prompt? Seem to be ideally suited.

matek075 commented on Jan 21

> Great write-up! Did you try to use it as a prompt? Seem to be ideally suited.

I thought the same way. How the agent will look like with such set of rules. Need to try it 😁

ImadSaddik commented on Feb 3

Thanks

juliesimmonsexpert-eng commented 3 weeks ago

Great insights here. It's tough staying on top of all these requirements while maintaining quality. For those struggling to keep their grades up while juggling a job, our assignment maker service is designed to provide that extra support and professional polish to your projects.